Reinforcement Learning on the HyMeKo Hypergraph

A cart-pole testbed, a vectorized PPO baseline, an honest structure ablation, and the path to storing a trained policy as data

Aiko (Claude Code) — for Dr. Csaba Hajdu

2026-06-21

Abstract

HyMeKo already expresses a Markov Decision Process *as data*: the robot (`meta_kinematics`), the scene (`meta_env`), the reward (`meta_reward`), and the exploration/optimisation strategy (`meta_strategy`) are all `.hymeko` descriptions compiled through one signed-incidence-hypergraph IR. This report documents the first end-to-end RL agent on that substrate: a HyMeKo-described inverted pendulum whose kinematic hypergraph is read directly by an HSiKAN actor–critic, trained by an in-repo PPO. We (i) build and validate the testbed, (ii) make the training loop 3.1× faster with a vectorized rollout after a measurement localised the cost, and (iii) run the discriminating control that *overturns* an attractive-but-wrong first conclusion: on the 2-vertex cart-pole the hypergraph structure is *not* load-bearing — a parameter-matched MLP ties the HSiKAN policy. We close with the storage thesis: HyMeKo can hold not only the MDP but the policy network's architecture and tensor shapes today (`meta_nn`), and — with a small, well-defined step — the trained weights, yielding a single content-addressed artifact that is simultaneously spec and model.

1 The MDP as data

The unification HyMeKo targets is "one source, many products." For control this means the entire MDP is a set of `.hymeko` profiles read by the trainer, with nothing about the task hard-coded in Python:

- **Robot** — `data/robotics/inverted_pendulum.hymeko`: a cart on a prismatic rail (`AXIS_X`) and a pole on a continuous hinge (`AXIS_Y`), on the `meta_kinematics` vocabulary. One emit yields the MuJoCo scene *and* the 2-vertex (cart, pole) signed kinematic hypergraph.
- **Environment / reward / strategy** — the companion `meta_env`, `meta_reward`, `meta_strategy` vocabularies (already used by the Galambos planar-grasp task).

The kinematic hypergraph is the object the structural policy reads: `HypergraphState.from_mjcf` derives the signed adjacency (A^+, A^-) over which the HSiKAN backbone message-passes.

2 Architecture: a shared-PPO, swappable-backbone actor–critic

The plan is *fix the algorithm, ablate the architecture*. One PPO loop trains either backbone; the two differ *only* in how they read the observation.

- **Observation** — per-vertex features $(N, 2) = [q, \dot{q}]$ on the kinematic hypergraph. Flattened this is the classic $[x, \dot{x}, \theta, \dot{\theta}]$, so both backbones read the same signal.
- **HSiKAN backbone** — signed message passing over (A^+, A^-) (a dense, batched SGCN variant), mean-pooled to a graph embedding.
- **MLP backbone** — the same observation flattened through a `tanh` MLP (the ablation baseline).
- **Actor–critic** — separate actor and critic backbones (so the value loss cannot corrupt the actor features); diagonal-Gaussian actor with a state-independent learned $\log \sigma$.

Algorithm. PPO: clipped surrogate, GAE, entropy bonus, and a time-limit *truncation bootstrap* ($+\gamma V(s')$ folded into the reward on a time-out, not a true terminal) — the fix that stops PPO degrading good policies. The loop depends only on a `RolloutEnv Protocol` (`reset`/ `step`), so the same trainer drives the pendulum or the reaching arm.

3 Engineering: a measurement-led 3.1× speedup

A diagnostic phase-timing of one iteration (not a guess) localised the cost: the batch-1 policy forward `ac.act` is **87%** of the rollout (2.84 s) vs `env.step` (0.41 s); the network is trivial (2 vertices), so the cost is per-step *torch dispatch overhead*, not FLOPs. A forward-vs-batch-size sweep confirmed dispatch-bound behaviour (per-call wall flat to B=8; per-sample 2.19 → 0.087 ms over B=1 → 32). The fix is a **vectorized rollout**: run N envs lock-step and batch the forward, so a rollout of `n_steps` transitions needs $\lceil$`n_steps` $/N\rceil$ ticks. Per-env GAE on each trajectory column; the truncation bootstrap is preserved *per env*.

	rollout	update	iter	120-iter wall
single-env	2.47 s	0.61 s	3.08 s	452.7 s
vec N=8	0.55 s	0.59 s	1.14 s	—
vec N=16	**0.36** s	0.57 s	**0.93** s	**147.4** s

Table 1: Vectorization: rollout 6.8×, full run 3.1×. Learning preserved (upright-steps 144 → 161). The single-env reach path is untouched; its regression suite passes.

A measurement-honesty note: a first end-to-end benchmark showed N=16 *slower* — an artifact of rebuilding N envs per measured round (each a `hymeko` CLI subprocess). Timing the real `_collect`/`_collect_vec`/`_update` and sharing one emitted MJCF exposed the true win.

4 Results and the discriminating control

All numbers: 5 seeds, vectorized (N=16), 120 iterations, upright-steps out of 200 (full-episode eval). Untrained floor ≈ 29 for every net.

net	params	upright (mean ± sd)	per-seed / learn rate	
HSiKAN	26 243	192.0 ± 15.3	161, 200, 199, 200, 200	5/5
MLP (original)	9 091	98.6 ± 74.7	39, 38, 182, 36, 198	2/5
MLP param-matched	**26 659**	**195.2 ± 8.4**	179, 197, 200, 200, 200	5/5
MLP over-param	134 659	200.0 ± 0.0	200×5	5/5

Table 2: Artifacts: `reports/2026-06-21-cartpole-{multiseed,controls}.jsonl`.

The honest finding. The first read — HSiKAN learns on 5/5 seeds while the original MLP is bimodal (2/5) — looked like a *structural robustness* win. The control overturns it: a *parameter-matched* MLP (26.7k ≈ HSiKAN's 26.2k) ties HSiKAN (195.2 ± 8.4, 5/5, lower variance), and an over-parameterised MLP is perfect. The original MLP's failures were **under-parameterisation** (hidden= 64), not the absence of structure. *On the 2-vertex cart-pole the kinematic-hypergraph structure is not load-bearing; capacity is.* This matches a prior on-record verdict (the 2026-06-18 rotor-vs-MLP-embed ablation): a matched-capacity control closed an apparent geometric-prior gap there too.

Scope. A 2-vertex graph has essentially no topology to exploit, so cart-pole *cannot* test the structure hypothesis — this is close to the expected result, not a refutation of structural policies

in general. The fair test needs a non-trivial graph: the 6-DOF arm-reach or the Galambos two-arm grasper. The methodological lesson, now a standing rule: *run the parameter-matched control before crediting structure.* The PPO *algorithm* baseline (192 ± 15) stands as the reference for future algorithms (DDPG/TD3/SAC); the *architecture* claim does not.

5 The storage thesis: from declarative MDP to stored policy

HyMeKo's reach for control extends past the environment. Three layers, at different maturity:

1. **MDP — already storage.** The whole cart-pole MDP is a `.hymeko` project.
2. **Policy architecture + tensor shapes — already storage.** `data/nn/meta_nn.hymeko` models networks as hypergraphs: tensors are vertices carrying `shape`; layers are hypervertices (`linear`, `signedkan_layer`, `walk_layer`, `arity_mixer`...) carrying GGK kernel specs $K = (B, G, \mu, r)$; `@dataflow` hyperedges wire ($+$in, $\sim$ layer, $-$out). `mnist_small.hymeko` is a full MLP that emits a torch `nn.Module` via the `transforms/torch_dataflow` backend. Tensors are not bolted on — the signed-incidence tensor *is* the core substrate.
3. **Trained weights — the gap.** `meta_nn` captures the shape *contract*, not the learned parameters. Closing it is the step from "spec" to "stored model." Three options, in increasing nativeness:
   (a) weights as `List`-valued fields on the tensor/layer vertices (the snapshot serializer now renders `List` values — plumbing exists; fine for small heads);
   (b) a content-addressed weight blob (`.safetensors`) keyed by the canonical-IR hash, so architecture and weights share one provenance stamp;
   (c) the structurally-honest one: because the network *is* a signed-incidence hypergraph and the learnable parameters are GGK coefficients on factors/edges, the weights live as edge/factor attribute tensors in the same hypergraph — one object, no second file.

The end state is a single `.hymeko` project that is simultaneously the environment spec, the network spec, and the trained model — content-addressed by one hash. For the cart-pole the only missing piece is describing the actor–critic in `meta_nn` (instead of the hand-coded backbone) and attaching its weights.

6 Conclusion and next steps

The testbed works, the loop is fast, and the architecture claim was correctly retired by its own control. Forward:

- **Algorithm breadth** — off-policy methods (DDPG, TD3, then SAC) for sample efficiency, with a *shared declarative exploration vocabulary* so PPO and DDPG differ only in where noise is injected and in the on-/off-policy core. (Design: the companion architecture & workflow plan.)
- **A task that can judge structure** — run the architecture ablation on the 6-DOF arm or Galambos, where the kinematic graph has real topology.
- **Policy storage** — prototype option (c): the cart-pole actor–critic as a `meta_nn` description with weights as factor-tensor fields — the first "MDP + trained policy, both in HyMeKo" artifact.

Per-task reports (full detail). `2026-06-21-cartpole-hsikan-wirein.md`, `2026-06-21-vectorized-ppo-rollou`
`2026-06-21-editor-{hyperedge-on-hyperedge,attribute-hud}.md`.